	Manhattan	Countdown	Path-Star	TinyStories	A_5	FineWeb-Edu
Training Parameters						
Steps	400k (6 epochs)	100k (204 epochs)	20k (51 epochs)	100k (8.5 epochs)	400k (512 epochs)	191k (100B tokens)
Batch Size	256	1024	512	256	1024	512
Learning Rate	1e-4	3e-4	5e-4	3e-4	1e-4	4e-4
Learning Rate Schedule	Constant	Constant	Constant	Constant	Constant	WSD
Weight Decay	0.01	0.1	0.1	0.1	0.01	0.1
Clip Gradient Norm	-	1	100	1	1	1
Optimizer	*AdamW* [Loshchilov and Hutter, 2019] with $\beta_1 = 0.9,\ \beta_2 = 0.95$					
Model Parameters						
Layers	48	12	12	8	2	22
Heads	8	12	6	8	8	16
Hidden Dimension	384	768	384	768	512	2048
NextLat Parameters						
$\lambda_{\text{next-h}}$	1.0	2.0	1.0	1.0	1.0	1.0
λ_{KL}	0.1	1.0	1.0	1.0	0.0	1.0
p_ψ MLP Hidden Dimension	1536	768	384	2048	1024	6528
p_ψ MLP Layers	3	3	3	3	3	3

Table 5: Training, Model, and NextLat hyperparameters across all benchmarks.

baseline comparisons. For FineWeb-Edu pretraining, extensive hyperparameter tuning is computationally prohibitive, so we use uniform loss weights across all methods (i.e., $\lambda_{\text{next-h}} = \lambda_{\text{KL}} = \lambda_{\text{MTP}} = 1.0$) to provide the fairest comparison possible under our compute constraints.

F.1 Manhattan Taxi Rides

Here, we provide additional details on our training and evaluation setups for the Manhattan Taxi Rides benchmark and clarify key differences from the original study of Vafa et al. [2024].

Training. Since this task inherently requires state tracking (i.e., tracking position within Manhattan), and increasing model depth is known to benefit transformers on such tasks [Merrill et al., 2025], we employ 48-layer transformers with 384 hidden dimensions and 8 attention heads (88M parameters). This differs from Vafa et al. [2024], which used 12 layers, 768 hidden dimensions, and 12 heads for their smaller transformer variant. We found that increasing model depth yielded substantial performance gains, whereas increasing hidden dimensionality offered negligible improvement. As shown in Table 1, the effective latent rank of our models is substantially smaller than 384, suggesting that large hidden dimensions are unnecessary.

Unlike the original study, which trained models for only one epoch, our models are trained for six epochs, as we observed that performances generally do not converge within a single epoch. This also helps rule out potential "grokking" phenomena [Power et al., 2022], where generalization improves only after extended periods of overfitting. To enable longer training without substantially increasing runtime, we apply sequence packing, i.e., concatenating multiple sequences into longer ones while masking cross-sequence attention. This enables efficient utilization of GPU memory and computation. Most models complete six training epochs in under three days on a single NVIDIA H100 NVL GPU, except MTP, which has substantially more parameters than the others.

Evaluation. Models are trained on random traversals of length 100 connected pickup and dropoff intersections. This task inherently requires not only state tracking but also planning, as models must reason over possible future paths to generate valid 100-step trajectories that reach the destination while avoiding dead ends, i.e., road segments disconnected from the goal due to the one-way streets. Random traversals rarely correspond to true shortest paths and do not provide an inductive bias toward learning the shortest path algorithm, which relies on dynamic programming. Consequently, unlike Vafa et al. [2024], who evaluated pairs with shortest paths of up to 100 steps, we limit evaluation pairs to paths of up to 50 steps. This adjustment ensures that evaluation pairs do not demand long-horizon planning beyond the training distribution, which might otherwise force the model to produce forced predictions to compensate for planning failure.

This also ensures that inconsistencies in a model's internal map are more reflective of world-model incoherence rather than artifacts of long-horizon planning limitations. These evaluation pairs are used to generate Figure 3 and to compute the sequence compression and detour robustness metrics, following the procedure of Vafa et al. [2024].

For the effective latent rank, we pass a batch of 256 sequences (each of length 256) through the model to obtain the hidden state matrix. Singular values smaller than $1e{-}12$ are discarded, and the effective rank is then computed following Roy and Vetterli [2007]. For GPT and NextLat, we use the final-layer hidden states. For JTP, we extract the hidden states immediately before the self-attention module in the Fetch head (see Equations 4–5 in Ahn et al. [2025]). For MTP, we use the output of the next-token prediction head to compute the effective rank.

F.2 Countdown

We largely follow Gandhi et al. [2024] for the Countdown training and evaluation setup. Each problem consists of four input numbers and a solution sequence comprising three equations, consistent with prior work [Gandhi et al., 2024, Ye et al., 2025]. A training example is formatted as

$$\underbrace{14, 83, 88, 91,}_{\text{inputs}} \overbrace{23}^{\text{target}} \mid \underbrace{83 - 14 = 69,\ 91 - 88 = 3,\ 69/3 = 23}_{\text{solution}}$$

where the first four numbers are the inputs, the fifth is the target number, and the pipe symbol " | " separates the input prompt from the solution. During training, loss values corresponding to input prompt are masked out.

Previous studies involving the Countdown benchmark used pretrained GPT-2 byte-pair encoding tokenizers, which do not necessarily tokenize multi-digit numbers as single units. In contrast, we construct a custom tokenizer that assigns each integer from 1 to 10,000 to a unique token, ensuring that every number in the sequence is represented atomically. The arithmetic operators and delimiters, i.e., $\{\ |\ ,\ +\ ,\ -\ ,\ \times\ ,\ \div\ \}$, are each assigned their own token indices. Due to the large branching factor of the Countdown problem, we insert eight pipe symbols (" | ") between the input and the solution as pause tokens [Goyal et al., 2023], allowing the model additional computation steps to plan before generating its answer.

F.3 Path-Star

Our Path-Star data preparation, training, and evaluation follow Bachmann and Nagarajan [2024], except that we increase the weight decay to 0.1, which we found helpful for stable convergence and higher solve rates in the multi-step prediction methods (i.e., MTP, JTP, and NextLat). We evaluate each model's ability to generate the correct arm on 20k held-out test instances. Unlike Hu et al. [2025] and Ahn et al. [2025] which generate a fresh set of graphs every batch, we adopt the original, more challenging setup of Bachmann and Nagarajan [2024], which uses a fixed sample size of 200k and node values sampled from $N = 100$. This difference accounts for the performance gap observed in the BST and JTP baselines in Figure 6. The Path-Star experiment is designed to expose the myopic behavior of teacher-forced next-token prediction, which can encourage models to exploit superficial regularities—an effect referred to as the *Clever Hans cheat* [Bachmann and Nagarajan, 2024]. Because the task's sample space grows exponentially with graph size, identifying the correct algorithm that generalizes across all graph instances is highly nontrivial. While not conclusive, our results suggest that latent-space prediction and the inductive bias toward compressing history into belief states promote better discovery of generalizable solutions in data-constrained settings.

F.4 TinyStories

Our TinyStories setup follows exactly Hu et al. [2025]. After pretraining on TinyStories, we train linear (one-layer) probes on the hidden states of the frozen transformer models for an additional 20k steps on the same dataset. All probe training hyperparameters (e.g., learning rate, batch size) match those used during pretraining (see Table 5).

For GPT, JTP, and NextLat, the choice of hidden states follows the setup used for measuring effective rank in the Manhattan taxi rides task, as described in Section F.1. For BST, we use the final-layer hidden states of the forward

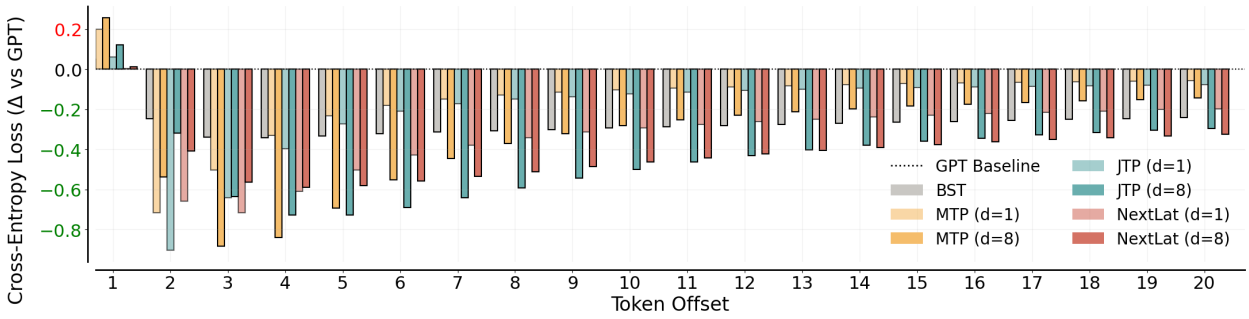


Figure 16: Full plot version of Figure 8 that shows probe performance across all 20 tokens offsets.

transformer encoder. For MTP [Gloeckle et al., 2024], we use the output of the shared transformer trunk, i.e., the hidden state before it branches into separate transformer heads for multi-token prediction, as this final shared representation contains the most predictive information about future tokens.

F.5 A_5 State Tracking

The group A_5 consists of the even permutations of 5 elements. Intuitively, each token in the sequence represents a permutation operation that rearranges the five elements, and after each operation the model must output the resulting arrangement (i.e., state). The task is therefore fundamentally a *state-tracking* problem: the model must maintain and update an internal representation of the current state as new permutation operations are sequentially composed.

Our training setup follows Merrill et al. [2025]. We trained the models on 1 million unique 12-token sequences over the group A_5. We then evaluated their length generalization capabilities on approximately 100k 36-token sequences. For NextLat training, we used **one-step supervision** ($d = 1$) and optimized only the regression objective ($\lambda_{\text{next-h}} = 1$), without token-level supervision ($\lambda_{\text{KL}} = 0$). This ensures that any observed length generalization of the latent dynamics model (RNN) arises purely from faithful next-latent prediction rather than auxiliary multi-token prediction signals.

We also experimented with transformers without positional embeddings (NoPE), motivated by prior work suggesting improved length generalization [Kazemnejad et al., 2023]. However, we found that NoPE degraded performance in this setting, so all experiments use Rotary Position Embeddings (RoPE; Su et al. [2023]). Unlike RNNs, whose hidden states evolve sequentially and therefore implicitly encode token order, transformers process sequences in parallel and rely heavily on positional embeddings to represent token order information during training. Interestingly, despite positional embeddings being explicitly included in the transformer hidden states, the latent dynamics model trained on one-step latent transitions derived from the transformer still generalizes beyond the training sequence length. This suggests that the learned transition dynamics is robust to the transformer's positional encodings.

F.6 FineWeb-Edu Pretraining

Following standard practices, all models are optimized with AdamW using a peak learning rate of $4e{-}4$, weight decay of 0.1, and gradient clipping at 1.0. We use a global batch size of 500M tokens and a sequence length of 1024 tokens. The learning rate follows a Warmup-Stable-Decay (WSD) schedule [Hu et al., 2024], consisting of a 1B-token linear warmup followed by a 10B-token linear decay phase. All models use the GPT-2 tokenizer with a vocabulary size of 50,257.

We evaluated the zero-shot accuracy of the pretrained models on nine standard language modeling benchmarks: Wikitext (Wiki; Merity et al. [2016]), LAMBADA (LAMB.; Paperno et al. [2016]) (standard version), PIQA [Bisk et al., 2020], HellaSwag (HellaS.; Zellers et al. [2019]), WinoGrande (Wino.; Sakaguchi et al. [2021]), ARC-easy (ARC-e) and ARC-challenge (ARC-c) [Clark et al., 2018], Social IQa (SIQA; Sap et al. [2019]), and SciQ [Welbl et al., 2017]. We also evaluate the self-speculative decoding performance of JTP, MTP, and NextLat across Wikipedia [Foundation],

Books (BookCorpusOpen; Face [2021]), Code (Stack-Edu; Allal et al. [2025]), and Math (OpenWebMath; Paster et al. [2023]) domains. To do so, we sample 1024 prompts of length 512 tokens from each dataset and generate 512-token continuations using the speculative sampling algorithm of Leviathan et al. [2022]. We then measured the speedup in inference of each model using self-speculative decoding compared to naive autoregressive sampling from the transformer, measured on 8×NVIDIA B200 GPUs.

F.6.1 Results with MTP/JTP (d=4)

Model	FW-Edu ppl ↓	Wiki. ppl ↓	LAMB. ppl ↓	LAMB. acc ↑	PIQA acc ↑	HellaS. acc ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc ↑	SIQA acc ↑	SciQ acc ↑	Avg.
GPT	**10.52**	**17.93**	20.26	42.07	73.45	**58.79**	60.46	68.18	39.16	42.32	86.10	58.82
JTP (d=1)	11.08	19.28	21.88	41.35	**74.92**	57.43	58.64	68.73	39.25	42.99	87.30	58.83
JTP (d=2)	11.18	19.60	22.11	41.37	73.34	56.84	59.98	68.86	38.57	**43.35**	86.70	58.63
JTP (d=4)	11.29	20.45	20.65	41.90	73.45	56.58	57.70	69.73	39.68	42.53	**88.50**	58.76
MTP (d=1)	10.90	18.82	20.23	41.26	74.32	58.05	**60.54**	68.52	38.91	42.84	85.40	58.76
MTP (d=2)	11.00	18.61	18.34	43.43	72.80	57.92	59.35	68.35	39.08	41.97	86.60	58.69
MTP (d=4)	11.10	18.97	22.75	40.69	73.72	57.39	58.33	**70.20**	39.51	42.12	85.90	58.48
NextLat (d=1)	10.83	18.39	19.77	41.08	73.07	58.35	59.27	69.65	39.68	43.24	86.00	58.79
NextLat (d=2)	10.88	18.44	**17.83**	**43.86**	73.61	57.79	59.20	69.74	**40.10**	41.91	87.50	**59.21**

Table 6: Downstream language modeling evaluation on 1.3B-parameter models trained on 100B FineWeb-Edu tokens. Best scores are in bold and second-best are underlined.

In this section, we extend our comparisons to include JTP and MTP trained with larger horizons ($d = 4$). Note that the training cost of JTP and MTP increases substantially with larger d (see Table 4), making JTP ($d = 4$) and MTP ($d = 4$) significantly more expensive to train than NextLat ($d = 1, 2$).

Table 6 shows that increasing the multi-token prediction horizon to $d = 4$ still does not yield meaningful improvements on multiple-choice benchmarks; NextLat ($d = 2$) continues to achieve the best average accuracy overall. Table 7 shows that even when JTP and MTP are trained to be able to draft more tokens (i.e., 4 tokens ahead), they still fail to surpass the variable-length speculative decoding performance of NextLat across most domains, with the exception of the Code domain. Finally, Figure 17 shows that, on the FineWeb-Edu validation set, increasing the training horizon to $d = 4$ yields only modest speedup gains for JTP and MTP. Their speedup curves saturate earlier and remain below that of NextLat ($d = 2$), which continues to improve with longer draft lengths and achieves the highest overall speedup.

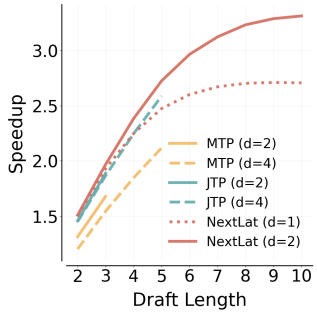


Figure 17: Inference speedup ratio on the FineWeb-Edu validation set.

Model	Wikipedia Speedup	Wikipedia Accepted Tokens	Books Speedup	Books Accepted Tokens	Code Speedup	Code Accepted Tokens	Math Speedup	Math Accepted Tokens
JTP (d=1)	1.46	0.96	1.47	0.97	1.47	0.98	1.46	0.97
JTP (d=2)	1.88	1.84	1.90	1.89	1.88	1.85	1.89	1.86
JTP (d=4)	2.58	3.32	2.62	3.43	**2.61**	3.43	2.42	3.02
MTP (d=1)	1.38	0.91	1.39	0.95	1.40	0.97	1.39	0.95
MTP (d=2)	1.68	1.72	1.72	1.83	1.75	1.91	1.72	1.84
MTP (d=4)	2.10	3.04	2.25	3.44	2.32	**3.68**	2.25	3.46
NextLat (d=1)	2.68	3.52	2.72	3.64	2.29	2.66	2.30	2.72
NextLat (d=2)	**3.21**	**4.59**	**3.32**	**4.86**	2.38	2.83	**2.87**	**3.94**

Table 7: Relative speedup and average accepted tokens per drafting steps over diverse domains. Note that "Accepted Tokens" excludes the next-token prediction which is always accepted.